strategynet.ai

Volt: Portfolio Control

Managing options as state, risk, and available actions change

RESEARCH ARCHITECTURE • VERSION 0.2

research@strategynet.ai

Open-source working paper • July 2026

ABSTRACT

Opening an options strategy is only the first decision. Each hedge, close, roll, exercise, assignment, and market move changes the position, its risk, and the actions that remain available. A policy trained on a static payoff table misses that path dependence; a policy allowed to redefine contract or accounting rules cannot be trusted.

This paper proposes a planning and reinforcement-learning layer that consumes a graph of the current portfolio, its relationships, candidate actions, and observed state. Volt exposes that structure to the learner while remaining responsible for contract meaning, fills, settlement, transitions, and replay. The public crate already provides the deterministic runtime, scenario paths, journals, a static portfolio graph, and a compiler-produced state/action graph with a hard structural legal mask. It does not yet provide account policy, a reward model, a trained policy, or a rollout scheduler. The implemented `cog` compiler constructs and coordinates the source, candidate, legal-action, and graph artifacts. QUBO is considered only as a possible rollout-budget allocator and would need to beat simpler schedulers under the same budget.

Contents

I Control Problem **3**
1 Portfolio state 3
2 Actions and masks 3
3 Rewards and horizons 3
II Learning and Search **5**
4 Policy architecture 5
5 Trajectory collection 5
6 Rollout allocation 5
6.1 Search frontier 5
6.2 QUBO proposal 6
6.3 Coverage and bias 6
7 Parallel execution 6
III Evaluation **7**
8 Protocol 7
9 Baselines 7
10 Evidence and boundary 7
11 Conclusion 7

PART I

Control Problem

1 Portfolio state

Static discovery asks which admissible portfolio looks attractive at one decision time. Portfolio control asks what to do next, observes the consequences, and asks again. Closing a short option may free capital but realize the spread; a hedge alters delta and financing; exercise consumes a right and may create cash or underlying inventory; assignment changes the account without being chosen by its owner. A useful state must carry these consequences forward rather than recomputing the position from a terminal payoff diagram.

In the control interface, `cog` emits

$$s_t = (G_t, A_t, M_t, p_t),$$

where G_t describes the current portfolio, account, market observations, and lifecycle state; A_t contains resolved actions; M_t records which actions are presently admissible; and p_t identifies the definitions, observations, assumptions, and prior journal state. The policy consumes this object but cannot edit its authority rules.

The first implementation combines these pieces. `PortfolioState` contains exact position lifecycle, cash, underlying units, sequence, and observation time; `CandidateGrammar` resolves bounded candidate actions; and `StateActionGraphSample` combines the static portfolio graph, runtime context, every accepted or rejected action slot, typed edges, and an aligned legal mask. The artifact enforces structural and lifecycle legality, not caller-owned account capital or margin policy.

A proposed environment transition binds an action to versioned state:

$$(s_{t+1}, r_t, e_t) = \mathcal{T}_\omega(s_t, a_t, \xi_{t+1}),$$

where $\mathcal{T}_\omega$ combines the Volt runtime with declared market and execution model ω, ξ_{t+1} contains the next market and external events, r_t is a derived reward record, and e_t is immutable transition evidence. The implemented runtime itself accepts a `PortfolioState`, `TransitionAction`, and `MarketObservation`, returning the next state and one journal entry. Replaying the same versioned inputs reproduces state and emissions.

2 Actions and masks

The implemented grammar contains `AddLeg`, `RemoveLeg`, `ResizeLeg`, `Wait`, and holder-authorized `Exercise`. The runtime action vocabulary is distinct: `Hold`, `Close`, `Exercise`, `Assign`, and `Expire`. The initial compiler does not yet turn close, roll, hedge, take-profit, or reduce-risk instructions into a multi-step runtime plan. Assignment remains an external event because a short writer does not possess the holder's exercise right.

Action resolution takes place at the decision clock. A semantic request such as "roll the short call to the next monthly expiration near 25 delta" must resolve only against contracts and observations visible at that time. The result names exact listed contracts and quantities. Volt enforces structural and exercise rules; an application must calculate estimated resource use and apply liquidity, position, and account constraints before a policy normalizes its scores.

Masks may change abruptly along a path. Expiration removes a contract; exercise consumes a right; a fill changes available cash; assignment creates inventory; a market halt may make an otherwise legal close unavailable under the execution model. These changes are part of the environment rather than exceptions handled in model code.

3 Rewards and horizons

A control environment may derive financial outcomes as a vector:

$$r_t = (\Delta\text{equity}, \text{cost}, \text{drawdown}, \text{capital}, \text{turnover}, \text{constraint events}).$$

The current simulator records fills, costs, journals, marked pathwise P&L, leg attribution, and terminal P&L. Drawdown, capital, turnover, and constraint-event components require additional named models. An experiment may scalarize its declared vector for policy optimization. A common form is

$$R_t = \Delta\text{equity}_t - \lambda_c \text{cost}_t - \lambda_d \Delta\text{drawdown}_t - \lambda_k \text{capital}_t,$$

with coefficients fixed in the manifest. Reward shaping requires particular care because a policy can optimize the shaping rule while degrading terminal wealth or tail exposure.

Economic and computational horizons should be named separately. The episode may end at expiration, liquidation, a risk breach, or the end of the historical window. Training can still use shorter n-step returns or truncated recurrent unrolls with a bootstrap value:

$$\widehat{R}_t^{(n)} = \sum_{j=0}^{n-1} \gamma^j R_{t+j} + \gamma^n \widehat{V}(s_{t+n}).$$

This truncation limits computation and credit-assignment length; it does not declare the financial position finished at $t + n$. The bootstrap state and all open obligations remain in the trajectory record.

PART II

Learning and Search

4 Policy architecture

A heterogeneous graph encoder maps G_t to a portfolio state h_t. Candidate actions receive embeddings $e(a)$, and a policy head scores each pair:

$$\ell_\theta(s_t, a) = f_\theta(h_t, e(a), h_t \odot e(a)) .$$

Masked logits define

$$\pi_\theta(a \mid s_t) = \mathrm{softmax}_{a \in A_t}\big(\ell_\theta(s_t, a) + \log M_t(a)\big),$$

using the implementation convention that excluded actions receive negative infinity before normalization. A value head estimates expected discounted return or its distribution. The encoder may be initialized from supervised strategy ranking, although control performance remains the criterion that determines whether the transfer was useful.

Policy-gradient, actor–critic, or value-based algorithms can consume this interface. The initial implementation should favor a well-understood baseline such as PPO with generalized advantage estimation [1, 2]; algorithmic novelty would make failures harder to locate while the environment and reward contract are still being tested.

5 Trajectory collection

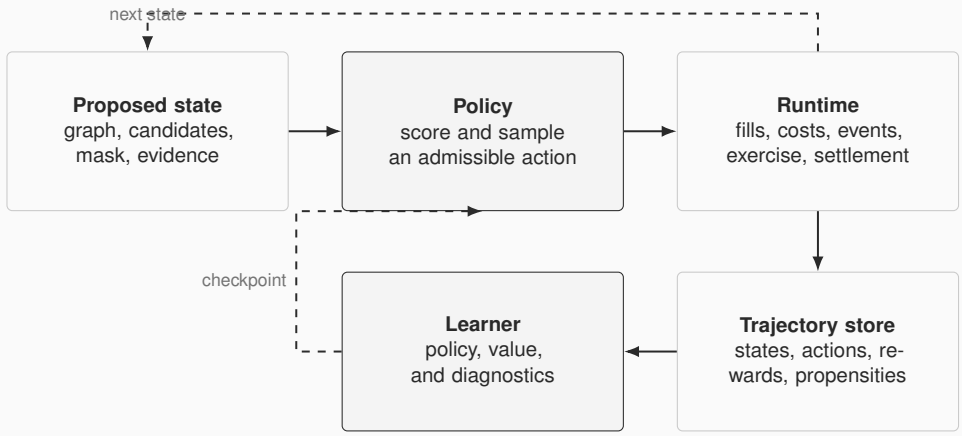


Figure 1. The proposed control boundary. Policy workers would choose from compiler masks; the runtime owns portfolio transitions and the learner would update from versioned trajectories.

Independent workers can collect trajectories for different dates, underlyings, scenario paths, execution assumptions, and random seeds. They share a frozen policy checkpoint during a collection epoch and emit immutable trajectory fragments. A deterministic merge orders fragments by stable identifiers before optimization, so worker completion order does not change the training batch.

The stored record includes the policy version and action probability used at collection time. Those fields matter when asynchronous workers lag the learner or when an allocation rule changes which experiences enter an update. Without them, off-policy drift and selection bias become difficult to measure or correct.

6 Rollout allocation

6.1 Search frontier

Game-playing systems demonstrate that a learned policy and value function can concentrate tree-search effort without simulating every continuation to a terminal state [3, 4]. An option-management tree has related structure but a different source of branching: market scenarios, external events, execution uncertainty, and portfolio actions all create descendants. The cost of one expansion can also vary sharply with the scenario model and remaining horizon.

An allocator receives a frontier of partial paths and a finite budget. It may decide which nodes receive another action expansion, another market scenario, or a longer continuation. Useful signals include policy entropy, value uncertainty, tail-risk uncertainty, novelty, estimated improvement, path probability, and simulation cost. A simple upper-confidence or tree-search rule should establish the baseline before a combinatorial allocator is added.

6.2 QUBO proposal

Let $x_i \in \{0, 1\}$ indicate whether frontier item i receives additional compute. For equal-cost expansions and cardinality budget k, consider

$$\min_x -\sum_i u_i x_i + \lambda \sum_{i<j} s_{ij} x_i x_j + \rho \left(\sum_i x_i - k \right)^2 .$$

The linear utility u_i can combine expected improvement, epistemic uncertainty, tail relevance, and path probability. Pairwise similarity s_{ij} penalizes allocating the whole batch to nearly identical portfolio states or market paths. The final term encodes the compute budget. Variable simulation costs require a weighted constraint and binary slack variables, or a separate constrained quadratic formulation.

This construction has a close published precedent. Kholodov, Salloum, Jnadi, Khubiev, and Petrenko formulate Monte Carlo episode selection as QUBO, using linear terms for high-return episodes and quadratic penalties for trajectory redundancy and poor state coverage [7]. Their evidence comes from a finite GridWorld and establishes feasibility of the selection mechanism. Portfolio control requires its own evidence.

QUBO can enter on either side of simulation. Frontier allocation occurs before simulation and can save runtime cost, but its utility estimates are necessarily incomplete. Replay selection occurs after collection, when returns and coverage are known; it saves learner compute while leaving rollout cost unchanged. Prioritized experience replay provides the natural non-QUBO comparison for the latter [5].

6.3 Coverage and bias

Aggressive selection can remove the paths that reveal delayed losses, rare assignment states, or a mistaken value estimate. A production-quality experiment therefore reserves part of every budget for random or stratified exploration and treats rejection as a decision for the current budget, never as proof that a legal path is irrelevant.

Every selected item records its score components, pairwise penalties, selection probability where estimable, solver configuration, and QUBO solution quality. Replay-based learning can use importance weights to reduce the bias introduced by nonuniform sampling. Frontier selection is harder: unexpanded descendants have no observed return, so evaluation must compare the resulting policy with exhaustive small trees and with fixed-budget baselines.

The quadratic similarity matrix grows as $O(n^2)$. Large frontiers will need clustering, sparse neighborhoods, or staged selection before QUBO construction. Solver time is charged to the same wall-clock budget as simulation; a sophisticated allocator that costs more than the rollouts it saves has failed its engineering objective.

7 Parallel execution

Scenario paths and environment instances form the main parallel dimension. Graph encoding and policy inference can be batched on a GPU, while homogeneous pricing or scenario kernels may share that device. Exercise, assignment, correction, and journal processing remain control-heavy and can stay on the CPU until profiling supports a different lowering.

Rollout scheduling may be hierarchical. A coordinator divides a global budget among market slices or scenario families; local allocators choose frontier nodes within each partition; workers execute the selected requests. Stable identifiers preserve the relationship between a partial path, its parent, the policy checkpoint, and the evidence returned by the runtime.

PART III

Evaluation

8 Protocol

Development begins with synthetic and bounded historical environments in which exhaustive or near-exhaustive rollout trees remain available. The protocol then expands along one dimension at a time: horizon, action grammar, scenario count, execution uncertainty, and number of contracts. This order makes it possible to attribute a performance change to a specific source of branching.

Train, validation, and test periods remain chronological. Reward coefficients, termination rules, truncation length, bootstrap convention, random exploration share, and rollout budget are frozen before test evaluation. Results are reported over multiple policy and environment seeds, including failures and constraint events.

9 Baselines

The control policy is compared with economically meaningful management rules: hold to expiration, fixed take-profit and stop rules, clock-based hedging, delta-threshold hedging, and simple roll conventions. Learning comparisons include uniform rollout allocation, random frontier expansion, beam search, UCT-style tree search [6], greedy utility, greedy utility with diversity, and prioritized replay. QUBO receives the same candidate frontier and total compute allowance.

Measurements cover:

- net and gross P&L, drawdown, tail loss, turnover, and capital use;
- frequency and severity of constraint or lifecycle failures;
- policy stability across costs, spreads, and scenario assumptions;
- state and action coverage, including assignment and exercise paths;
- value calibration and bootstrap error by remaining horizon;
- simulator calls, expanded nodes, learner updates, and wall-clock time;
- allocation overhead and solution quality for each scheduler.

Equal simulator counts answer a statistical-efficiency question; equal wall-clock budgets answer an engineering question. Both are required because parallel collection, GPU batching, and QUBO solution overhead change the relationship between those budgets.

10 Evidence and boundary

The policy should choose only actions authorized by the compiler and composed application mask. The scenario layer supplies declared close fills and fees; the runtime validates exact contract actions, handles exercise and assignment, updates cash and inventory, and journals the transition. The learner owns scores, action probabilities, value estimates, advantages, and checkpoint updates. Rollout allocation owns compute scheduling. These records meet through stable state and request identifiers without sharing authority.

The repository contains no quote-backed historical policy result. A touch fill remains a declared execution convention, and a path selected after observing its return cannot be described as a tradable policy. Economic evidence requires an untouched period, capacity and cost analysis, sensitivity to market and assignment models, and comparison with simple management rules.

11 Conclusion

Volt supplies the deterministic state and transition core from which a sequential-control environment can be built while contract semantics remain stable. That boundary can let research move from PPO to tree search or QUBO-guided scheduling without changing what an option, exercise, fill, or assignment means. The immediate research goal is a small replayable environment with exhaustive controls; only after its learning and accounting agree should rollout allocation be asked to save compute on larger trees.

References

[1] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, *High-Dimensional Continuous Control Using Generalized Advantage Estimation*, International Conference on Learning Representations, 2016, https://arxiv.org/abs/1506.02438.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, *Proximal Policy Optimization Algorithms*, 2017, https://arxiv.org/abs/1707.06347.

[3] D. Silver et al., *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*, 2017, https://arxiv.org/abs/1712.01815.

[4] J. Schrittwieser et al., *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model*, Nature, 588, pp. 604–609, 2020, https://doi.org/10.1038/s41586-020-03051-4.

[5] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, *Prioritized Experience Replay*, International Conference on Learning Representations, 2016, https://arxiv.org/abs/1511.05952.

[6] L. Kocsis and C. Szepesvári, *Bandit Based Monte-Carlo Planning*, European Conference on Machine Learning, 2006, https://doi.org/10.1007/11871842_29.

[7] Y. A. Kholodov, H. Salloum, A. Jnadi, K. Yu. Khubiev, and A. Petrenko, *Quantum-Inspired Episode Selection for Monte Carlo Reinforcement Learning via QUBO Optimization*, Computer Research and Modeling, 18(2), pp. 273–288, 2026, https://doi.org/10.20537/2076-7633-2026-18-2-273-288.